

PROYECTO 2

Aplicación con APIs Públicas

Diseño y Desarrollo de Sistemas • Licenciatura en Ciencias de la Computación

Campo	Detalle
Materia	Diseño y Desarrollo de Sistemas
Carrera	Licenciatura en Ciencias de la Computación
Modalidad	Grupal (2–4 integrantes)
Entrega final	A definir por la cátedra
Lenguaje / Plataforma	Kotlin — el mismo stack del Proyecto 1

1. Introducción y Objetivo

En el Proyecto 1 aplicaron ingeniería inversa sobre un código base existente, lo refactorizaron hacia una arquitectura limpia (Clean MVVM), incorporaron tests unitarios y extendieron el sistema integrando una segunda API. Todo ese trabajo sirvió para internalizar patrones, principios y herramientas sobre un dominio acotado.

El Proyecto 2 les propone dar el paso siguiente: construir desde cero una aplicación cuyo dominio eligen ustedes. El objetivo es el mismo — producir software bien arquitecturado, testeable y mantenible — pero ahora tienen libertad creativa para decidir qué problema resolver y qué APIs públicas consumir.

La arquitectura, los tests y el cumplimiento de SOLID y Clean Code no son opcionales: son el eje central de la evaluación. Una app con features llamativas pero con arquitectura pobre o sin tests no aprueba el proyecto.

2. Descripción General

Deberán diseñar y desarrollar una aplicación original que consuma **al menos dos APIs públicas gratuitas** e integre sus datos para aportar valor al usuario.

La aplicación debe contar con al menos dos pantallas con navegación entre ellas, manejar correctamente los estados de error y carga, y persistir datos localmente para funcionar (aunque sea parcialmente) sin conexión a internet.

3. Propuestas de Proyectos (no excluyentes)

Las siguientes ideas son orientativas. Pueden adoptarlas tal cual, modificarlas o proponer algo completamente diferente, siempre que el scope sea comparable y utilicen al menos dos APIs.

3.1 Dashboard del Clima + Astronomía

Combinar datos meteorológicos en tiempo real con información astronómica para mostrar condiciones climáticas, fase lunar, hora del amanecer/ocaso y eventos del espacio cercano.

- API sugerida 1: Open-Meteo (clima, sin clave requerida)
- API sugerida 2: NASA Astronomy Picture of the Day / NeoWs (asteroides)
- Pantallas mínimas: home con clima actual + datos astronómicos del día; historial de búsquedas por ciudad

3.2 Explorador de Noticias Temáticas

Aplicación que permite al usuario suscribirse a categorías de noticias, leerlas, guardarlas como favoritas y traducir titulares a otro idioma.

- API sugerida 1: NewsAPI (noticias por categoría y fuente)
- API sugerida 2: MyMemory Translation API (traducción gratuita, sin clave)
- Pantallas mínimas: feed de noticias; detalle de noticia; favoritos guardados localmente

3.3 Guía de Países del Mundo

App educativa que muestra información detallada de países: datos geográficos, económicos, idiomas y banderas, con posibilidad de comparar dos países lado a lado.

- API sugerida 1: RestCountries (datos de países, sin clave)
- API sugerida 2: Open-Meteo o OpenWeatherMap (clima actual de la capital del país)
- Pantallas mínimas: listado de países con búsqueda/filtro; detalle del país; pantalla de comparación

3.4 Tracker de Criptomonedas

Dashboard personalizable que permite al usuario seguir cotizaciones en tiempo real, ver tendencias de precio y recibir información de mercado relevante.

- API sugerida 1: CoinGecko (precios, volumen, market cap — sin clave para el tier gratuito)
- API sugerida 2: NewsAPI (noticias del mercado cripto filtradas por keyword)
- Pantallas mínimas: listado de monedas con precios; detalle con gráfico de tendencia; watchlist persistida localmente

3.5 App de Recetas con Plan Nutricional

Buscador de recetas por ingredientes o categoría, que además calcula información nutricional estimada y permite armar un plan semanal de comidas.

- API sugerida 1: TheMealDB (recetas, ingredientes, categorías — completamente gratuita)
- API sugerida 2: Open Food Facts API (información nutricional de ingredientes)
- Pantallas mínimas: búsqueda de recetas; detalle con ingredientes y pasos; plan semanal guardado localmente

3.6 Asistente de Viajes

Aplicación que permite planificar un viaje: explorar países de destino, consultar el clima esperado, y guardar un itinerario personal.

- API sugerida 1: RestCountries (información del destino: moneda, idioma, zona horaria)
- API sugerida 2: Open-Meteo (pronóstico extendido para el destino)
- Pantallas mínimas: explorador de destinos; detalle del destino con clima; mis viajes guardados

3.7 Perfil Gamer Steam

Aplicación que permite al usuario explorar su biblioteca de Steam, visualizar estadísticas de juego personales y descubrir información editorial enriquecida de cada título combinando datos de dos fuentes.

- API sugerida 1: Steam Web API (biblioteca del usuario, horas jugadas, logros, noticias — requiere API key gratuita en steamcommunity.com/dev/apikey)
- API sugerida 2: RAWG Video Games Database (metadatos editoriales, Metacritic score, géneros, screenshots — gratuita, sin clave para el tier básico)
- Pantallas mínimas: búsqueda de perfil por SteamID; lista de biblioteca con filtro por género y ordenamiento por horas jugadas; detalle del juego con datos combinados de ambas APIs (logros propios + información editorial); watchlist de juegos favoritos persistida localmente

3.8 Proyecto Libre

Si tienen una idea propia, pueden proponerla. Debe cumplir con:

- Uso de al menos dos APIs públicas gratuitas
- Scope comparable al Proyecto 1 (al menos 2 pantallas, datos combinados de múltiples fuentes, persistencia local)
- Aprobación previa de la cátedra antes de comenzar el desarrollo

4. APIs Públicas de Referencia

La siguiente tabla no es exhaustiva. Pueden usar cualquier API pública gratuita que no requiera un plan de pago para el scope del proyecto.

Domínio	API Sugerida	URL / Docs
Clima / Geolocalización	Open-Meteo	open-meteo.com/en/docs
Clima / Geolocalización	OpenWeatherMap (free tier)	openweathermap.org/api
Astronomía / Espacio	NASA APIs	api.nasa.gov
Noticias	NewsAPI (free tier)	newsapi.org
Libros / Literatura	Open Library (Internet Archive)	openlibrary.org/developers/api
Música	MusicBrainz API	musicbrainz.org/doc/MusicBrainz_API
Recetas / Cocina	TheMealDB	themealdb.com/api.php
Ejercicio / Fitness	ExerciseDB (RapidAPI free)	rapidapi.com/justin-WFnsXH_t6/api/exercisedb
Criptomonedas / Finanzas	CoinGecko	coingecko.com/en/api
Diccionario / Idiomas	Free Dictionary API	dictionaryapi.dev
Países / Geografía	RestCountries	restcountries.com
Anime / Manga	Jikan (MyAnimeList unofficial)	jikan.moe
Cómics	Marvel Comics API (free key)	developer.marvel.com
Trivia / Juegos	Open Trivia DB	opentdb.com/api_config.php
Transporte / Viajes	Amadeus for Developers (sandbox)	developers.amadeus.com
Videojuegos / Gaming	Steam Web API (Valve)	steamcommunity.com/dev
Videojuegos / Gaming	RAWG Video Games Database	rawg.io/apidocs

5. Requisitos Técnicos Obligatorios

5.1 Arquitectura — Clean Architecture + MVVM

El proyecto debe respetar la misma separación de capas trabajada en el Proyecto 1:

- Presentation: Composables, ViewModels (solo depende de Domain)
- Domain: Use Cases, Entidades, interfaces de repositorios (sin dependencias externas)
- Data: Implementación de repositorios, fuentes remotas (Retrofit) y local (Room o DataStore)
- DI: Módulo de inyección de dependencias (Hilt o manual con factories)

Regla fundamental: las capas internas (Domain) no pueden depender de las capas externas (Data, Presentation). Cualquier violación de esta regla implica corrección obligatoria.

5.2 Clean Code + SOLID

- Nombres expresivos: clases, funciones y variables comunican su intención sin necesidad de comentarios.
- Funciones cortas y con una sola responsabilidad (regla de las 20 líneas como guía).
- Sin código muerto: no dejar clases, funciones ni imports sin usar.
- Sin magic numbers ni strings hardcodeados: usar constantes con nombre.
- Consistencia: el mismo estilo en todo el proyecto (nomenclatura, estructura de paquetes, convenciones Kotlin).

5.3 Tests Unitarios

Deben implementar tests unitarios para las siguientes clases, con cobertura de los happy paths y edge cases relevantes:

- ViewModels (todos)
- Use Cases (todos)
- Implementación del repositorio
- Data Sources locales
- Broker / lógica de combinación de datos entre APIs (si aplica)

5.5 Manejo de Estados en UI

La UI debe reflejar correctamente los estados Loading, Success y Error. No está permitido mostrar pantallas en blanco o crashear frente a respuestas vacías o errores de red.

*El objetivo del proyecto no es solo que la app funcione, **sino que sea un software del que puedan estar orgullosos: bien diseñado, probado y mantenible.***
